Tasks

Victor Eijkhout

2026 PCSE

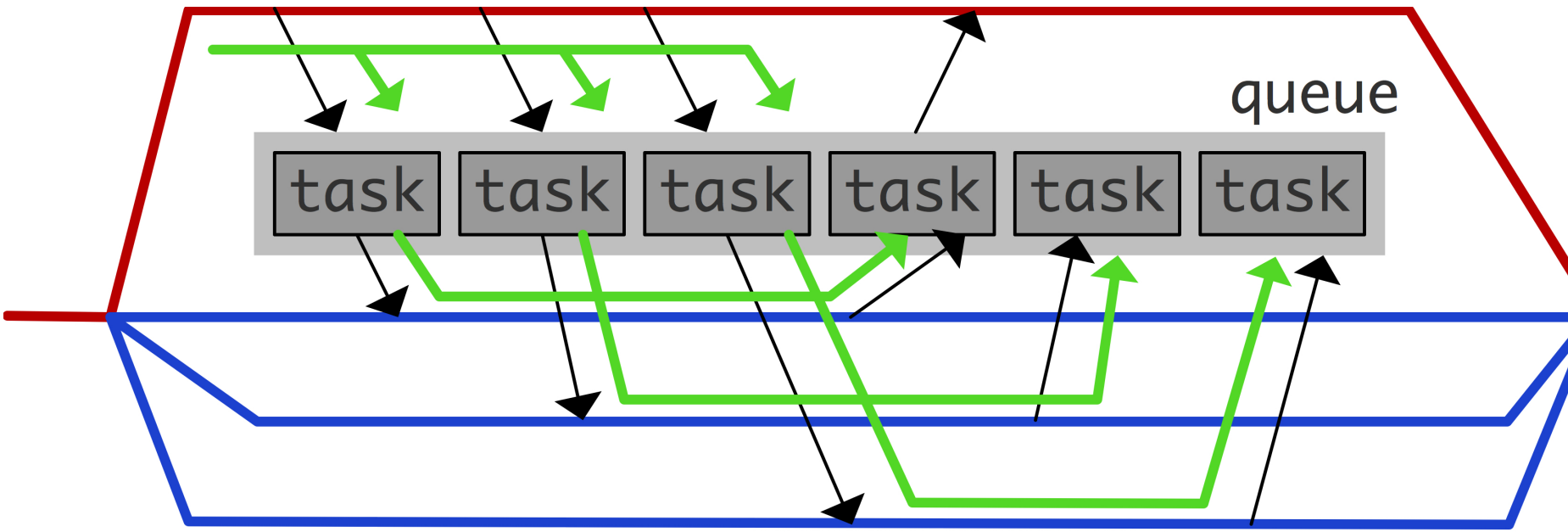
Tasks

- Loops generate 'implicit tasks'
  need static number of iterations
- Also: 'explicit tasks'
  can deal with dynamic work:
- Example linked lists or trees
- Tasks are very flexible:
  you create work, it goes on a queue, gets executed later

```
1 p = head_of_list();
2 while (!end_of_list(p)) {
3 #pragma omp task
4   process( p );
5   p = next_element(p);
6 }
```

Threads, tasks, queues

- There is one queue (per team), not visible to the programmer.
- One thread starts generating tasks.
- Tasks can recursively generate tasks.
- You never know who executes what.



Generation idiom

```
1 #pragma omp parallel
2 #pragma omp single
3 {
4   ...
5   #pragma omp task
6   { ... }
7   #pragma omp taskwait
8   ...
9 }
```

(Complicated logic of scheduling points; hardly ever matters)

Recursive generation

```
1  void f() {
2    do_something();
3    #pragma omp task
4      do_task1();
5    #pragma omp task
6      do_task2();
7    #pragma omp taskwait
8    do_something_more();
9  }
10 ...
11 #pragma omp parallel
12 #pragma omp single
13 #pragma omp task
14   f();
```

- Any thread can create tasks
- including threads that are executing tasks

Exercise 1 `taskfactor`

Use tasks to find the smallest factor of a large number (using $2999 \cdot 3001$ as test case): generate a task for each trial factor.

- Turn the factor finding block into a task.
- Run your program a number of times:

```
1 for i in $( seq 1 1000 ) ; do
2     ./taskfactor
3 done | grep -v 2999
```

  Does it find the wrong factor? Why? Try to fix this.

- Once a factor has been found, you should stop generating tasks. Let tasks that should not have been generated, meaning that they test a candidate larger than the factor found, print out a message.

Task data space

- Private data of the parallel region is captured as firstprivate unless explicitly specified otherwise.
- Makes sense: execution is deferred

```
1 #pragma omp parallel
2 #pragma omp single
3 for ( int i /* ... */ )
4   #pragma omp task
5   f(i);
```

  so you need value when the task was created.
- Shared data has to be captured explicitly

Task data: tricky exception

Shared data of the parallel region is shared in the task; if you want firstprivate, specify explicitly:

```
1 #pragma omp parallel shared(i)
2 #pragma omp single
3 for ( i /* ... */ ) // note: no 'int'
4   #pragma omp task firstprivate(i)
5   f(i);
```

Deferred and undeferred tasks

- Tasks are unusally 'deferred': they are executed at some undetermined point in the future.
- Tasks can also be undeferred: executed here and now.
- Prime example: with `if` clause

```
#pragma omp task if (level>5)
```

Task synchronization

Mechanisms for task synchronization:

- `taskwait`: wait for all previous tasks (not nested)
- `taskgroup`: wait for all tasks, including nested
- `depend`: synchronize on data items.

Task wait

Insider parallel region:

```
1 #pragma omp task
2 ...
3 #pragma omp task
4 ...
5 #pragma omp taskwait
```

- Wait on tasks spawned directly
- Not 'descendant' tasks

Task group

```
1 #pragma omp taskgroup
2 {
3   #pragma omp task
4   foo(0);
5   #pragma omp task
6   for ( int i=1; i<N; ++i )
7     #pragma omp task
8     foo(i)
9 }
```

- Wait for tasks on this level
- . . . and tasks created by tasks on this level

Example: tree traversal

```
1 int process( node n ) {
2   if (n.is_leaf)
3     return n.value;
4   for ( c : n.children) {
5     #pragma omp task
6     process(c);
7 #pragma omp taskwait
8   return sum_of_children();
9 }
```

Exercise: Fibonacci

Fix the errors in this program.

```
1  long fib(int n) {
2    if (n<2) return n;
3    else { long f1,f2;
4      #pragma omp task
5      f1 = fib(n-1);
6      #pragma omp task
7      f2 = fib(n-2);
8      return f1+f2;
9    }
10 }
11
12 #pragma omp parallel
13 #pragma omp single
14   printf("Fib(50)=%ld",fib(50));
```

Task cancelling

#pragma omp cancel construct [if (expr)]

where construct is parallel, sections, do or taskgroup.

More cancelling

Need to set *OMP_CANCELLATION* explicitly.

Set more cancellation points:

#pragma omp cancellation point <construct>

Task dependencies

```
1 #pragma omp task
2   x = f()
3 #pragma omp task
4   y = g(x)
```

```
1 #pragma omp task depend(out:x)
2   x = f()
3 #pragma omp task depend(in:x)
4   y = g(x)
```

Sequence tasks by indicating

- The output of one task
- the needed input of another
- (only between sibling tasks)

Complicated dependencies

Multiple dependencies:

```
1 #pragma omp task depend(in:x,y)
2
3 #pragma omp task depend(in:x) depend(out:y)
```

Dependencies on array sections:

```
1 #pragma omp task depend(in:x[1:n−1) depend(out:y[:m])
```

Exercise 2

Assume input files *input0*,*input1*,..., each containing a single integer. The following code reads these integers, transforms them into another integer, and then writes out the results to files *output0*,*output1*,...:

```
1 for ( int ifile=0; ifile<nfiles; ifile++ ) {
2   inputs[ifile] = read_input(ifile);
3   outputs[ifile] = transform( inputs,ifile );
4   write_output( ifile,outputs[ifile] );
5 }
```

With the `transform` function defined as:

```
1 int transform( int *inputs,int ifile ) {
2   int output = 0;
3   for (int iifile=0; iifile<=ifile; iifile++ )
4     output += inputs[iifile];
5   return output;
6 }
```

Use a task for each of the three lines in the loop body. Use dependencies to ensure correct synchronization.

Exercise 3

```
1 for i in [1:N]:
2     x[0,i] = some_function_of(i)
3     x[i,0] = some_function_of(i)
4
5 for i in [1:N]:
6     for j in [1:N]:
7         x[i,j] = x[i-1,j]+x[i,j-1]
```

- Use tasks
- Is there a different way to do this?

Fibonacci memo'ization

```
1  long fibvalues[100];
2  void fibonacci(n) {
3    if (n>=2) {
4      #pragma omp task \
5         depend( in:fibvalues[n-2],in:fibvalues[n-1] ) \
6         depend( out:fibvalues[n] )
7      fibvalues[n] = fibvalues[n-2]+fibvalues[n-1];
8  };
9
10 #pragma omp parallel
11 #pragma omp single
12   for (i<50)
13     fibonacci(i);
```

Taskloop

```
1 #pragma omp taskloop grainsize(5)
2 for ( int i=0; i<N; ++i )
3   f(i)
```

- Turn iterations into tasks
- Implicit taskgroup, unless *nogroup* specified